GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 13 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Strings and parsing

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	1
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Strings and parsing | 4 questions | tap a title to jump to it

M	1. Can board states be transformed?	PracHub	2026-04-01
M	2. Decide string transform with directional tokens and walls	PracHub	2025-07-15
M	3. Snake Case to Camel Case with Repository-Wide Refactor	1Point3Acres	2026-07-14
H	4. Find the Longest Straight Run of One Color	PracHub	2026-08-23

STUDY PATH: question 69 of 167 (Core 90)

Priority score 2.61 | Strings and parsing | asked 202 days ago

MEDIUM | PracHub | Technical Screen | 1 of 4

Can board states be transformed?

POSTED 2026-04-01 | 202 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 93/228

ABOUT THIS QUESTION

Topic: Core 90 - Strings and parsing

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (inferred from overview)

Posted on PracHub: 2026-04-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/can-board-states-be-transformed

QUESTION

Overview: This question evaluates reasoning about constrained piece movement, string transformation, and invariant-based correctness in the Coding & Algorithms domain, testing skills in string manipulation and combinatorial reasoning while examining both conceptual properties and practical algorithmic implementation.

APPROACH

Invariants: the sequence of non-empty pieces must match; each L can only move left and each R only right (compare positions with two pointers).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the pieces are, which direction each may move, whether pieces may jump over each other, and what the empty cell is called.

BRUTE FORCE TO SAY OUT LOUD

BFS over all reachable board strings. Exponential but clear.

OPTIMAL IDEA

Invariants: pieces never cross, so the order of the non-empty pieces must be identical in both strings; an L can only move left, so in the start its index must be at least its index in the target; an R can only move right, so its start index must be at most its target index. Walk two pointers over the non-empty cells and check those conditions. O(n).

DRY-RUN THESE CASES

Equal strings. A different piece order. An L that would have to move right. Same pieces but different counts.

LIKELY FOLLOW-UPS

Add walls that cannot be passed or crossed, or ask for the minimum number of moves.

PITFALLS

INTERVIEW-DAY PLAN

Comparing counts only and not order. Skipping the empty cells incorrectly in the two-pointer loop.

HOW TO THINK ABOUT IT

Think invariants, not search. Pieces cannot jump each other, so the sequence of L and R letters (ignoring blanks) cannot change. Within that, an L can only go left and an R only right, so compare the position of each piece in the start and in the target. That reduces exponential BFS to one pass with two pointers.

Basis: LeetCode 777, Swap Adjacent in LR String: 'X' is an empty cell, 'L' only moves left, 'R' only moves right, pieces never pass each other.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in legal_moves(board):
            if nxt not in seen:  # XL->LX, RX->XR
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential in the length	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially. Pieces never cross, so compare the sequence of L and R pieces and check that no L moved right and no R moved left, in O(n).

OPTIMAL SOLUTION · Two pointers over the pieces

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the minimum number of moves.

Because pieces never interact beyond keeping their order, each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences when the transformation is possible.

FOLLOW-UP 2 Add walls that cannot be crossed (next question).

A wall is just another piece that never moves: it stays in the sequence of non-blank symbols and must keep the same index in both strings. The same pointer walk handles it.

FOLLOW-UP 3 Pieces can move one step per round and several move at once.

The count of rounds is the maximum over pieces of their distance, if pieces never block each other; with blocking you simulate rounds or use the per-piece arrival times with the ordering constraint.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Can board states be transformed?
#
# This question evaluates reasoning about constrained
# piece movement, string transformation, and invariant-
# based correctness in the Coding & Algorithms domain,
# testing skills in string manipulation and combinatorial
# reasoning while examining both conceptual properties and
# practical algorithmic implementation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the minimum number of moves.
# 2. Add walls that cannot be crossed (next question).
# 3. Pieces can move one step per round and several move
# at once.
#
# INTERVIEW NOTES
# Think invariants, not search. Pieces cannot jump each
# other, so the sequence of L and R letters (ignoring
# blanks) cannot change. Within that, an L can only go
# left and an R only right, so compare the position of
# each piece in the start and in the target. That reduces
# exponential BFS to one pass with two pointers.
#
# BRUTE FORCE (say it first): BFS over every reachable
# board
# From the start string, repeatedly apply the legal moves
# (XL becomes LX, RX becomes XR) and see whether the
# target appears. Correct, and the oracle for the
# invariant-based check, but the number of boards
# explodes. Time exponential in the length. Space
# exponential.


from collections import deque


# BRUTE FORCE (say it first): BFS over every reachable
# board | time exponential in the length
def can_transform_bfs(start, end):
    seen, queue = {start}, deque([start])
    while queue:
        s = queue.popleft()
        if s == end:
            return True
        for i in range(len(s) - 1):
            nxt = None
            if s[i : i + 2] == "XL":
                nxt = s[:i] + "LX" + s[i + 2 :]
            elif s[i : i + 2] == "RX":
                nxt = s[:i] + "XR" + s[i + 2 :]
            if nxt and nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False


# HELPERS for the base question
def pieces(board):
    # the non-blank symbols with their positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "X"]


# BASE QUESTION (optimal): Two pointers over the pieces
# Idea: Skip blanks in both strings and compare the next
# piece in each: they must be the same letter.
def can_transform(start, end):
    # L moves left, R moves right, X is blank. Pieces
    # never pass
    # each other, so the order of the non-X symbols is
    # fixed;
    # a '#' is a wall: it never moves and no piece crosses
    # it.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        # an L may only end up further left, an R further
        # right
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
        # a wall must stay exactly where it is
        if c1 == "#" and i != j:
            return False
    return True


# FOLLOW-UP 1: Return the minimum number of moves.
# Answer: Because pieces never interact beyond keeping
# their order, each piece moves independently from its
# start index to its target index, so the minimum is the
# sum of the absolute index differences when the
# transformation is possible.
def min_moves(start, end):
    # every piece moves on its own, one cell per move
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    return sum(abs(i - j) for (_, i), (_, j) in zip(a, b))


# FOLLOW-UP 3: Pieces can move one step per round and
# several move at once.
# Answer: The count of rounds is the maximum over pieces
# of their distance, if pieces never block each other;
# with blocking you simulate rounds or use the per-piece
# arrival times with the ordering constraint.
def rounds_needed(start, end):
    # if all pieces move at once and never block each
    # other,
    # the number of rounds is the longest single trip
    if not can_transform(start, end):
```

```
        return None
    a, b = pieces(start), pieces(end)
    trips = [abs(i - j) for (_, i), (_, j) in zip(a, b)]
    return max(trips, default=0)


if __name__ == "__main__":
    assert can_transform("RXXLRXRXL", "XRLXXRRLX")
    assert not can_transform("X", "L")
    assert can_transform("XXXL", "XXLX")
    # R can not move left, L can not move right
    assert not can_transform("XR", "RX")
    assert not can_transform("LX", "XL")
    # pieces keep their order: RL can not become LR
    assert not can_transform("RL", "LR")
    assert min_moves("RXXLRXRXL", "XRLXXRRLX") == 4
    assert min_moves("RXX", "XXR") == 2
```

```
    assert min_moves("RX", "XR") == 1
    assert min_moves("LL", "RR") is None
    # a wall splits the board: R can not jump over it
    assert can_transform("R#X", "R#X")
    assert not can_transform("R#X", "X#R")
    assert rounds_needed("RXX", "XXR") == 2
    assert rounds_needed("LL", "RR") is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(start: str, target: str) -> bool:
```

Sample call

```
solution("R_L|__L", "_RL|L__")
```

Test cases

Input	Expected
("R_L|__L", "_RL|L__")	True
("L_|R", "L|_R")	False

STUDY PATH: question 70 of 167 (Core 90)

Priority score 2.61 | Strings and parsing | asked 462 days ago

MEDIUM | PracHub | Technical Screen | 2 of 4

Decide string transform with directional tokens and walls

POSTED 2025-07-15 | 462 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 94/228

ABOUT THIS QUESTION

Topic: Core 90 - Strings and parsing

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (with X walls)

Posted on PracHub: 2025-07-15 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/decide-string-transform-with-directional-tokens-and-walls

QUESTION

You are given two equal-length strings start and target of length n, each consisting only of the characters 'L', 'R', '.', and 'X'. A dot '.' represents an empty slot; 'X' represents an immovable wall; 'L' and 'R' are tokens that may move across empty slots subject to these rules: (1) An 'L' token may move any number of positions left into '.', but may never move right, pass through another token, or pass through an 'X'. (2) An 'R' token may move any number of positions right into '.', but may never move left, pass through another token, or pass through an 'X'. (3) Only one token can occupy a position at any time. Determine whether target can be obtained from start by a sequence of valid moves. If yes, return true; otherwise, return false. Design an algorithm that runs in O(n) time and O(1) extra space beyond a few pointers/counters. Explain your invariants, prove correctness informally, analyze time and space complexity, and walk through a few tricky edge cases (e.g., mismatched wall layouts, blocks separated by walls, consecutive tokens with no gaps).

APPROACH

Split both strings at 'X' walls (must align), then within each segment apply the L/R two-pointer rule: L only moves left, R only right, order of pieces preserved.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what each symbol means (L moves left, R moves right, dot empty, X wall that never moves and cannot be crossed), whether a token can jump over another (no), and whether both strings have the same length.

BRUTE FORCE TO SAY OUT LOUD

BFS over all board states by moving tokens one step. Exponential.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Compare the sequences of non-dot characters: they must be identical, which also forces every X to appear in the same order. Then with two pointers over those tokens check that each L has start index at least target index, each R has start index at most target index, and each X has the same index in both. Walls cannot be crossed because order is preserved. O(n).

DRY-RUN THESE CASES

Identical strings, an L that would move right, an R blocked by a wall, a different number of tokens, a wall at a different index.

LIKELY FOLLOW-UPS

Return the minimum number of moves (sum of index differences), or the move sequence.

PITFALLS

Letting a token cross a wall. Checking only counts of L and R.

HOW TO THINK ABOUT IT

Walls are pieces that never move. Ignore the dots and compare the sequences of L, R and X: they must match in order and in count, and each X must be at the same index in both strings. For the L and R pieces keep the same rule: L never moves right, R never moves left. Order is preserved, so no piece can get across a wall.

Basis: LeetCode 777 with walls: 'L' moves left, 'R' moves right, '.' is empty, 'X' is an immovable wall; no piece crosses another piece or a wall.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in one_step(board):
            if nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially with the length. Comparing the non-dot sequence (walls and pieces in order) with position rules (L never moves right, R never moves left, walls stay) is O(n).

OPTIMAL SOLUTION · Compare the non-dot sequences with position rules

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Segments between walls are independent.

You can also split both strings at the walls (they must be at identical positions) and apply the L/R rule to each segment separately. The single pass above is equivalent.

FOLLOW-UP 2 Return the moves or the minimum number of steps.

Per piece, the steps are the index difference; list moves by letting each piece walk one cell at a time in an order that never blocks (move L pieces left to right and R pieces right to left).

FOLLOW-UP 3 The board is 2D.

Then pieces can pass each other by going around, and invariants like order no longer hold. Ask for the exact movement rules; most likely you would model it as a graph search with a heuristic.

FOLLOW-UP 4 Return the minimum number of moves.

When the transformation is possible each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences (checked against BFS on small strings).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Decide string transform with directional
# tokens and walls
#
# You are given two equal-length strings start and target
# of length n, each consisting only of the characters 'L',
# 'R', '.', and 'X'. A dot '.' represents an empty slot;
# 'X' represents an immovable wall; 'L' and 'R' are tokens
# that may move across empty slots subject to these rules:
# ( 1) An 'L' token may move any number of positions left
# into '.', but may never move right, pass through another
# token, or pass through an 'X'. ( 2) An 'R' token may
# move any number of positions right into '.', but may
# never move left, pass through another token, or pass
# through an 'X'. ( 3) Only one token can occupy a
# position at any time. Determine whether target can be
# obtained from start by a sequence of valid moves. If
# yes, return true; otherwise, return false. Design an
# algorithm that runs in O(n) time and O( 1) extra space
# beyond a few pointers/counters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Segments between walls are independent.
# 2. Return the moves or the minimum number of steps.
# 3. The board is 2D.
# 4. Return the minimum number of moves.
#
# INTERVIEW NOTES
# Walls are pieces that never move. Ignore the dots and
# compare the sequences of L, R and X: they must match in
# order and in count, and each X must be at the same index
# in both strings. For the L and R pieces keep the same
# rule: L never moves right, R never moves left. Order is
# preserved, so no piece can get across a wall.
#
# BRUTE FORCE (say it first): BFS over boards with walls
# Apply the legal single-step moves ('.L' becomes 'L.',
# 'R.' becomes '.R'), walls never move, and search for the
# target. Exact, so it is the oracle for the pointer
# check, but it explodes with the length. Time
# exponential. Space exponential.


from collections import deque
```

```
# BRUTE FORCE over real moves (checker)
def bfs_moves(start, end):
    # brute force over real moves (the check used by the
    # demo)
    seen, queue = {start: 0}, deque([start])
    while queue:
        s = queue.popleft()
        if s == end:
            return seen[s]
        for i, ch in enumerate(s):
            for step, piece in ((-1, "L"), (1, "R")):
                j = i + step
                inside = 0 <= j < len(s)
                if ch == piece and inside and s[j] == ".":
                    t = list(s)
                    t[i], t[j] = ".", ch
                    t = "".join(t)
                    if t not in seen:
                        seen[t] = seen[s] + 1
                        queue.append(t)
    return None


# HELPERS for the base question
def pieces(board):
    # everything except the empty '.' slots, with
    # positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "."]


# BASE QUESTION (optimal): Compare the non-dot sequences
# with position rules
# Idea: Walk both strings skipping dots. The next symbol
# must be equal; a wall must be at the same index; an L
# must satisfy start index >= target index; an R must
# satisfy start index <= target index.
def can_transform(start, end):
    # 'L' moves left, 'R' moves right, 'X' is an immovable
    # wall. Pieces never pass each other, so their order
    # is
    # fixed; each one only has a direction it may move.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
```

```
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        if c1 == "X" and i != j:
            return False
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
    return True


# MIN MOVES (pieces move independently)
# FOLLOW-UP 4: Return the minimum number of moves.
# Answer: When the transformation is possible each piece
# moves independently from its start index to its target
# index, so the minimum is the sum of the absolute index
# differences (checked against BFS on small strings).
def min_moves(start, end):
    # pieces move independently: the sum of index
    # differences
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    return sum(abs(i - j) for (_, i), (_, j) in zip(a, b))


if __name__ == "__main__":
    assert can_transform("R.L", ".RL") is True
    assert not can_transform("RL.", "LR.")
    # a wall cannot move and splits the board
    assert can_transform("R.X.L", ".RX.L") is True
    assert not can_transform("R.X", "X.R")
    assert min_moves("R..L", ".RL.") == 2
    assert min_moves("L.", ".L") is None
    import itertools
    boards = []
    for n in range(1, 5):
        for p in itertools.product("LR.X", repeat=n):
            boards.append("".join(p))
    for s in boards:
        for t in boards:
            if len(s) == len(t) and len(s) <= 4:
```

```
                fast = min_moves(s, t)
                assert fast == bfs_moves(s, t), (s, t)
```

FUNCTION AND TEST CASES

Signature

```
def canTransform(start: str, target: str) -> bool:
```

Sample call

```
canTransform('R...L', 'RL...')
```

Test cases

Input	Expected
`('R...L', 'RL...')`	`True`
`('R.L', 'R..L')`	`False`

STUDY PATH: question 71 of 167 (Core 90)

Priority score 2.5 | Strings and parsing | asked 98 days ago | frequency Single report

MEDIUM | 1Point3Acres | tech-screen | 3 of 4

Snake Case to Camel Case with Repository-Wide Refactor

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 213/228**

ABOUT THIS QUESTION

Topic: Core 90 - Strings and parsing

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

APPROACH

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

HOW TO THINK ABOUT IT

Converting foo_bar to fooBar is easy. The risk is in the migration: a plain text replace also changes strings, comments, third-party names and public interfaces. Work on tokens (or the syntax tree) instead, rename only symbols defined in your own code, keep an explicit allow-list of public names, and prove nothing broke by running the tests before and after.

Basis: Written for this question; the conversion is a one-liner and the real work is a safe, token-based migration (not plain text replace).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for path in source_files(repo):
    text = read(path)
    for word in find_snake_words(text):
        if not is_third_party(word):
            new = to_camel(word)
            text = text.replace(word, new)
    write(path, text)
```

TIME	SPACE
O(size of the repository)	O(1)

WHY IT IS SLOW

Also rewrites strings, comments, third-party calls and public interface names. Renaming only the identifiers the code defines, found with the syntax tree and tokens, keeps those intact.

OPTIMAL SOLUTION · Convert names, rename only defined identifiers via tokens

TIME	SPACE
O(size of the source)	O(names)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you prove the migration is safe?

Run the full test suite before and after (green both times), compare the public API surface (exported names) before and after, and migrate in small reviewable commits with a codemod that only touches owned symbols. Anything reachable by reflection, serialisation, config files or other repositories needs an explicit alias or a deprecation period rather than a silent rename.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Snake Case to Camel Case with Repository-
# Wide Refactor
#
# Convert a valid snake_case identifier to camelCase, then
# design a safe repository-wide migration for variables
# and function names without changing third-party symbols,
# public interfaces, or the tests' ability to detect
# regressions. Implement a conversion from a valid
# snake_case identifier to camelCase. The base input is
# guaranteed valid: no leading underscore, trailing
# underscore, or doubled separator. Example: foo_bar ->
# fooBar. Extend the discussion to a repository too large
# for manual review: replace snake-case variables and
# functions across the codebase. Explain how to verify
# that the repository still works after the automated
# rewrite, including when test-package identifiers are
# rewritten too. Preserve third-party functions and public
# interfaces; explain how the migration distinguishes
# symbols that may change from contracts that must remain
# stable. Examples: foo_bar -> fooBar
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you prove the migration is safe?
#
# INTERVIEW NOTES
# Converting foo_bar to fooBar is easy. The risk is in the
# migration: a plain text replace also changes strings,
# comments, third-party names and public interfaces. Work
# on tokens (or the syntax tree) instead, rename only
# symbols defined in your own code, keep an explicit
# allow-list of public names, and prove nothing broke by
# running the tests before and after.
#
# BRUTE FORCE (say it first): Plain text replace across
# the files
# Regex-replace every snake_case word in every file. Fast
# and wrong: it rewrites strings, comments, third-party
# calls and public interface names. Time O(size of the
# repository). Space O(1).


import re
import ast
import io
import tokenize
```

```
# BRUTE FORCE (say it first): Plain text replace across
# the files | time O(size of the repository)
def snake_rewrite_brute(source):
    return re.sub(
        r"\b[a-z]+(?:_[a-z0-9]+)+\b",
        lambda m: snake_to_camel(m.group()),
        source,
    )


# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
# Idea: snake_to_camel capitalises each part after the
# first. owned_names collects the functions, arguments and
# variables the code defines (using the syntax tree),
# minus an allow-list of names that must stay.
def snake_to_camel(name):
    # foo_bar -> fooBar (input is a valid snake_case name)
    head, *rest = name.split("_")
    caps = [p[:1].upper() + p[1:] for p in rest]
    return head + "".join(caps)


# FOLLOW-UP 1: How do you prove the migration is safe?
# Answer: Run the full test suite before and after (green
# both times), compare the public API surface (exported
```

```
# names) before and after, and migrate in small reviewable
# commits with a codemod that only touches owned symbols.
def owned_names(source, keep=()):
    # names this code defines itself: safe to rename
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
            args = node.args.args + node.args.kwonlyargs
            names.update(a.arg for a in args)
        elif isinstance(node, ast.Name):
            if isinstance(node.ctx, ast.Store):
                names.add(node.id)
    # only real snake_case, never dunder or private names
    return {n for n in names if "_" in n.strip("_")
            and not n.startswith("_") and n not in keep}
```

```
# FOLLOW-UP 1: How do you prove the migration is safe?
# Answer: Run the full test suite before and after (green
# both times), compare the public API surface (exported
# names) before and after, and migrate in small reviewable
# commits with a codemod that only touches owned symbols.
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    out, prev = [], None
    # work on tokens: strings and comments are untouched
    for tok in tokenize.generate_tokens(
            io.StringIO(source).readline):
        # skip attribute names (after a dot): they may be
        # third-party, so they stay as written
        dot = prev is not None and prev.string == "."
        is_name = tok.type == tokenize.NAME
        if is_name and tok.string in owned and not dot:
            new = snake_to_camel(tok.string)
            tok = tok._replace(string=new)
        out.append(tok)
        prev = tok
    return tokenize.untokenize(out)


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    src = (
        "import os\n"
        "def load_data(file_path):\n"
        "    full_path = os.path.join(file_path, 'x_y')\n"
        "    return full_path  # load_data is fine\n"
        "result_value = load_data('a')\n")
    new = rename_identifiers(src, keep={"file_path"})
    assert "def loadData(file_path)" in new
    assert "fullPath = os.path.join" in new
    # strings, comments and third-party calls are intact
    assert "'x_y'" in new and "# load_data is fine" in new
    assert "resultValue = loadData('a')" in new
    # the migrated code still runs
    scope = {}
    exec(compile(new, "x", "exec"), scope)
    assert scope["resultValue"] == "a/x_y"
```

NOTES FROM CANDIDATES

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

PREPARATION

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

STUDY PATH: question 72 of 167 (Core 90)

Priority score 4.78 | Strings and parsing | asked 58 days ago

HARD | PracHub | Technical Screen | 4 of 4

Find the Longest Straight Run of One Color

POSTED 2026-08-23 | 58 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 19.6** **30 DAYS 0** **PRIORITY 145/228**

ABOUT THIS QUESTION

Topic: Core 90 - Strings and parsing

Optimal answer from: the interview site's own solution

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix

Posted on PracHub: 2026-08-23 (updated 2026-09-10)

Original: https://prachub.com/coding-questions/find-the-longest-straight-run-of-one-color

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **R** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

QUESTION

The down-left diagonal from the top-right cell contains three **`G`** cells. No horizontal or vertical sequence has that length.

APPROACH

Per-cell DP over four directions (horizontal, vertical, diagonal, anti-diagonal): run[d][i][j] = run[d][prev]+1 if same color else 1. O(m*n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether diagonals count in both directions (yes, anti-diagonals too), whether cells must share an exact color value, and what to return for an empty grid. Confirm that a run cannot bend or skip a cell.

BRUTE FORCE TO SAY OUT LOUD

For every cell and each of the four directions, walk forward while the color matches. That is O(m * n * min(m, n)) and is a fine first answer to say out loud.

OPTIMAL IDEA

Let run[d][i][j] be the length of the same-color run that ends at (i, j) going in direction d. It is run[d][prev] + 1 if the previous cell in that direction has the same color, else 1. Four directions need only the previous row, so memory can be O(n). Answer is the maximum over everything. Time O(m * n).

DRY-RUN THESE CASES

A single cell (answer 1). A 1 x n row of one color. A grid where the longest run is on the anti-diagonal. A grid where every neighbor differs (answer 1).

LIKELY FOLLOW-UPS

Return the cells of the run, not just the length. Allow one wildcard cell. Make the run length at least K and count how many such runs exist.

PITFALLS

Forgetting the anti-diagonal needs the up-right previous cell, so it must be filled in the right scan order. Counting a bent path as a run.

HOW TO THINK ABOUT IT

A run in a fixed direction depends only on the cell before it in that direction, so each of the four directions is a one-line recurrence (like longest common suffix). Only four directions are needed because a line read backwards is the same line. Rolling only the previous row keeps memory at O(n).

Basis: Written; the same recurrence as LeetCode 562, Longest Line of Consecutive One in Matrix, with any color instead of 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 1
for r, c in all_cells(grid):
    for dr, dc in DIRECTIONS:  # 4 directions
        n = 1
        while same(grid, r, c,
                   r + n * dr, c + n * dc):
            n += 1
        best = max(best, n)
```

TIME	SPACE
O(m * n * min(m, n))	O(1)

WHY IT IS SLOW

A long run is re-walked from every cell inside it, O(m * n * min(m, n)). A table of the run length ending at each cell per direction is O(m * n).

OPTIMAL SOLUTION · DP per direction with a rolling row

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step back by its length.

FOLLOW-UP 2 Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

FOLLOW-UP 3 The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the Longest Straight Run of One
# Color
#
# Given a rectangular grid of colors, return the length of
# the longest consecutive sequence of cells with the same
# color along a horizontal, vertical, or diagonal straight
# line. A sequence cannot change direction or skip a cell.
# Both diagonal orientations count. The length is the
# number of cells in the sequence. ### Input - grid: an
# array of strings of equal length. Each character
# represents one cell's color. ### Output Return the
# maximum length as an integer. ### Constraints and Edge
# Cases - For this practice version, the grid contains at
# least one row and one column, with at most 1000000 cells
# in total. - Colors are uppercase English letters; equal
# letters mean equal colors. - A one-cell grid has answer
# 1. - Diagonal lines may run down-right or down-left.
# Reading the same line in reverse does not change its
# length. - A connected region that bends is not
# necessarily one valid sequence.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the run is, not only its length.
# 2. Require the run to be at least K and count how many
# such runs exist.
# 3. The grid is huge and arrives row by row.
#
# INTERVIEW NOTES
# A run in a fixed direction depends only on the cell
# before it in that direction, so each of the four
# directions is a one-line recurrence (like longest common
# suffix). Only four directions are needed because a line
# read backwards is the same line. Rolling only the
# previous row keeps memory at O(n).
#
# BRUTE FORCE (say it first): From every cell, walk in
# each of four directions
# For each start cell and direction, step while the color
# matches. Correct and short, but cells in a long run are
# re-walked from every start. Time O(m * n * min(m, n)).
# Space O(1).


# BRUTE FORCE (say it first): From every cell, walk in
# each of four directions | time O(m * n * min(m, n))
DIRS4 = [(0, 1), (1, 0), (1, 1), (1, -1)]


def longest_run_brute(grid):
    best = 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            for dr, dc in DIRS4:
                k = 0
                while (
                    0 <= r + dr * k < len(grid)
                    and 0 <= c + dc * k < len(grid[0])
                    and grid[r + dr * k][c + dc * k]
                    == grid[r][c]
                ):
                    k += 1
                best = max(best, k)
    return best


# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


def runs(grid):
    # yield (length, start, direction) for every maximal
    # run;
    # a run is walked only from its first cell, so the
    # total
    # work is linear in the number of cells
    rows, cols = len(grid), len(grid[0])
```

```
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                pr, pc = r - dr, c - dc
                before = 0 <= pr < rows and 0 <= pc < cols
                if before and grid[pr][pc] == grid[r][c]:
                    # not the first cell of its run
                    continue
                n, a, b = 0, r, c
                while 0 <= a < rows and 0 <= b < cols \
                        and grid[a][b] == grid[r][c]:
                    n += 1
                    a, b = a + dr, b + dc
                yield n, (r, c), (dr, dc)
```

```
# BASE QUESTION (optimal): DP per direction with a rolling
# row
# Idea: run[d][i][j] is the length of the same-color run
# ending at (i, j) in direction d. It is the previous
# cell's run plus one when the colors match, else 1.
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    return max(n for n, _, _ in runs(grid))


# FOLLOW-UP 1: Return where the run is, not only its
# length.
# Answer: Only start a walk at cells that begin a run (the
# previous cell in that direction has a different color);
# that keeps the total work linear, and you record the
# start cell and direction of the best run.
def longest_run_where(grid):
    # the length, the start cell and the direction
    if not grid or not grid[0]:
        return 0, None, None
    return max(runs(grid), key=lambda x: x[0])


# FOLLOW-UP 2: Require the run to be at least K and count
# how many such runs exist.
# Answer: Count a run once, at its start, when its length
# is at least K (use the begins-a-run test above).
# Counting at every cell would count a long run many
# times.
def count_runs_at_least(grid, k):
    # each run is counted once, at its start
    if not grid or not grid[0]:
        return 0
    return sum(1 for n, _, _ in runs(grid) if n >= k)


# FOLLOW-UP 3: The grid is huge and arrives row by row.
# Answer: The rolling-row version already needs only the
# previous row, so it works on a stream of rows with O(n)
# memory.
def longest_run_rows(row_stream):
    # rows arrive one by one: keep only the previous row's
```

```
    # runs
    best, prev = 0, None
    h = v = d = a = None
    for row in row_stream:
        n = len(row)
        nh, nv = [1] * n, [1] * n
        nd, na = [1] * n, [1] * n
        for c in range(n):
            if c and row[c] == row[c - 1]:
                nh[c] = nh[c - 1] + 1
            if prev is not None:
                if row[c] == prev[c]:
                    nv[c] = v[c] + 1
                if c and row[c] == prev[c - 1]:
                    nd[c] = d[c - 1] + 1
                if c + 1 < n and row[c] == prev[c + 1]:
                    na[c] = a[c + 1] + 1
            best = max(best, nh[c], nv[c], nd[c], na[c])
        prev, h, v, d, a = row, nh, nv, nd, na
    return best


if __name__ == "__main__":
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_run(g1) == 4
    assert longest_run(["RBG", "BGR", "GRB"]) == 3
    assert longest_run(["R"]) == 1
    assert longest_run([]) == 0
    length, start, direction = longest_run_where(g1)
    assert length == 4
    # runs of length at least 3 are counted once each
    assert count_runs_at_least(g1, 3) == 4
    assert longest_run_rows(iter(g1)) == 4
```

FUNCTION AND TEST CASES

Signature

```
def longestColorRun(grid: list[str]) -> int:
```

Sample call

```
longestColorRun(['RRRG', 'RRBB', 'GRRR', 'BGBR'])
```

Test cases

Input	Expected
(['RRRG', 'RRBB', 'GRRR', 'BGBR'],)	4
(['RBG', 'BGR', 'GRB'],)	3

ALSO REPORTED · 1 more

Longest Same-Color Grid Sequence | 1Point3Acres | phone-screen | asked 2026-09-08 | Single report

https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

ALSO REPORTED · 1 more

Preparation: Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.